iSMART

Циклы в Python

Как заставить программу повторять действия автоматически

Урок: Данные и алгоритмическое мышление (8–11 класс)



iSMART

Ручной труд (Долго)

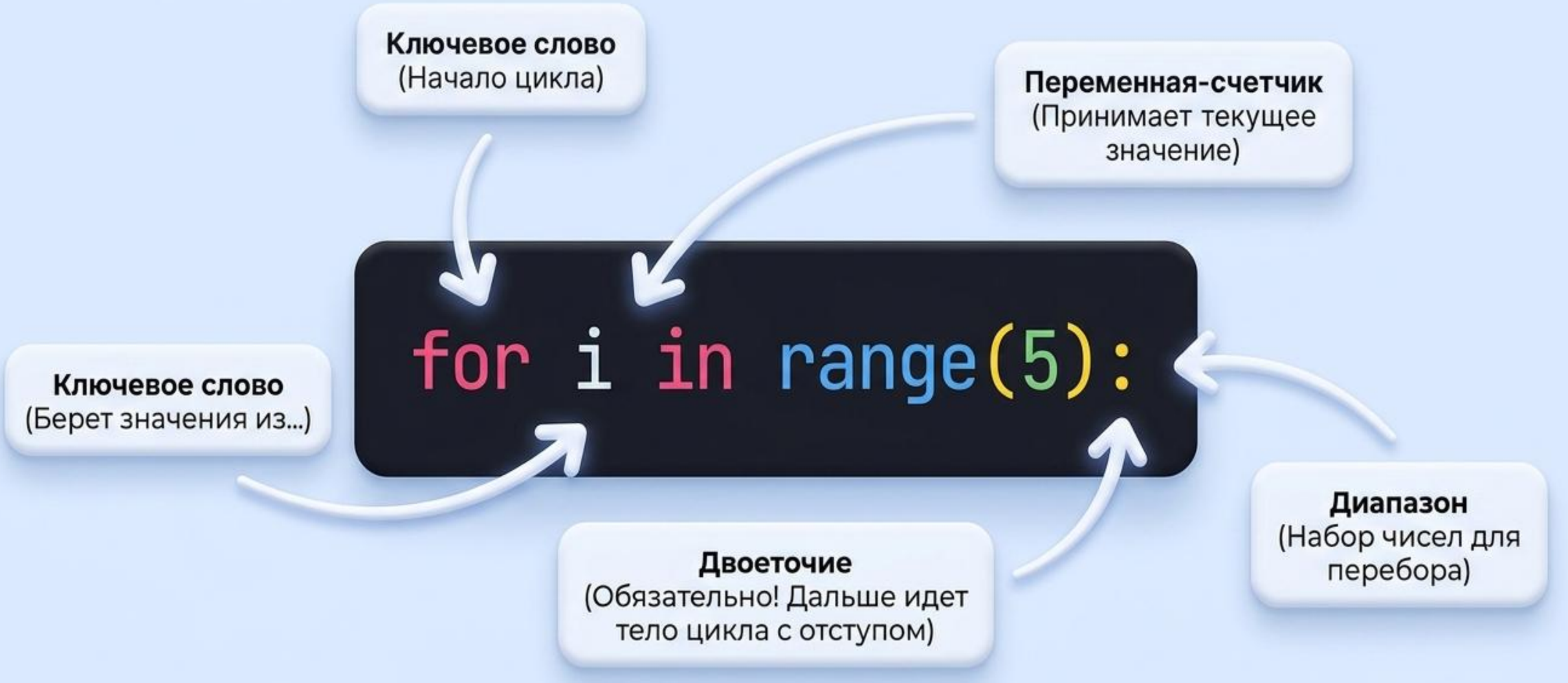
```
print("Привет!")
print("Привет!")
print("Привет!")
print("Привет!")
print("Привет!")
print("Привет!")
... (ещё 97 раз)
```

Магия циклов (Быстро)

```
for i in range(100):
    print("Привет!")
```



Без циклов не работает ни один калькулятор, ни одна игра, ни один поисковик!

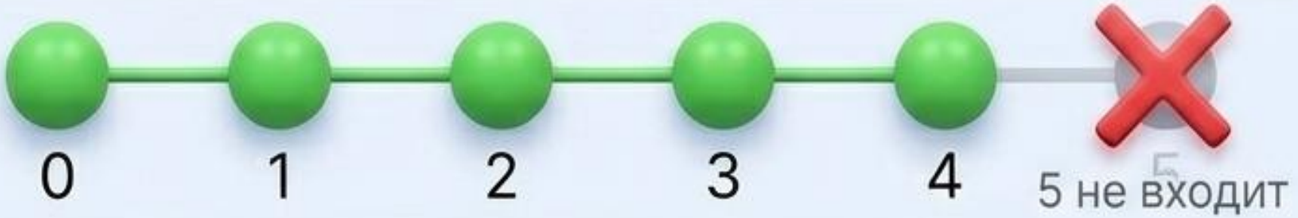


Цикл for используется, когда заранее известно, сколько раз нужно повторить действие.

iSMART

iSMART

`range(5)`



`range(1, 6)`



`range(1, 10, 2)`
(от 1 до 9, шаг 2)



`range(10, 0, -1)`
(обратный отсчет)



iSMART

```
total = 0

for i in range(1, 4):
    total = total + i
```

Как меняются переменные?

Старт	→		total = 0
Шаг 1	→	i = 1	→ total = 0 + 1 = 1
Шаг 2	→	i = 2	→ total = 1 + 2 = 3
Шаг 3	→	i = 3	→ total = 3 + 3 = 6



После завершения цикла total равно 6.

Цикл while: Повторение по условию

```
number = int(input("Введите 0
для остановки: "))

while number != 0:
    print("Работаем!")
    number = int(input("Введите
число: "))
```



Используется, когда неизвестно, сколько раз пользователь будет вводить данные.

Осторожно: Бесконечный цикл!

```
x = 1
while x <= 4:
    print("Привет!")
    # ОШИБКА: x никогда не меняется!
```



Программа зависнет! Всегда проверяйте, что условие внутри while когда-нибудь станет ложным.

```
x = x + 1
```

(Добавьте шаг, чтобы выйти из цикла!)

iSMART

Что выбрать: for или while?

Когда использовать for



- ✅ Известно точное количество повторений (например, 10 раз).
- ✅ Нужно перебрать диапазон чисел.

Пример: Вывести таблицу умножения.

Когда использовать while



- ✅ Повторять, пока выполняется условие.
- ✅ Повторять до ввода определенного значения (стоп-слова).

Пример: Считать числа, пока не введут ноль.

Счетчик (Counter)



Зачем нужен?
Считает количество событий (например, сколько четных чисел).

Старт:

```
count = 0
```

Обновление внутри цикла:

```
count = count + 1
```

Накопитель (Accumulator)



Зачем нужен?
Суммирует значения (например, общая сумма покупок).

Старт:

```
total = 0
```

Обновление внутри цикла:

```
total = total + x
```

Максимум и Минимум (Maximum & Minimum)

Максимум (Maximum)



Зачем нужен?
Запоминает самое большое число из набора.

Старт: max_val = 0 (или первое введенное значение)

Условие обновления:

```
if x > max_val:
    max_val = x
```

Минимум (Minimum)



Зачем нужен?
Запоминает самое маленькое число.

Старт: min_val = первое введенное значение

Условие обновления:

```
if x < min_val:
    min_val = x
```

Управление циклом: оператор break

Немедленно прерывает цикл, даже если он еще не закончен.




```
while True:
    x = int(input("Введите число: "))
    if x == 0:
        break  # Экстренный выход!
```

Идеально подходит для выхода при нахождении нужного значения.



Управление циклом: оператор continue

Пропускает остаток тела цикла и сразу переходит к следующему шагу (итерации).

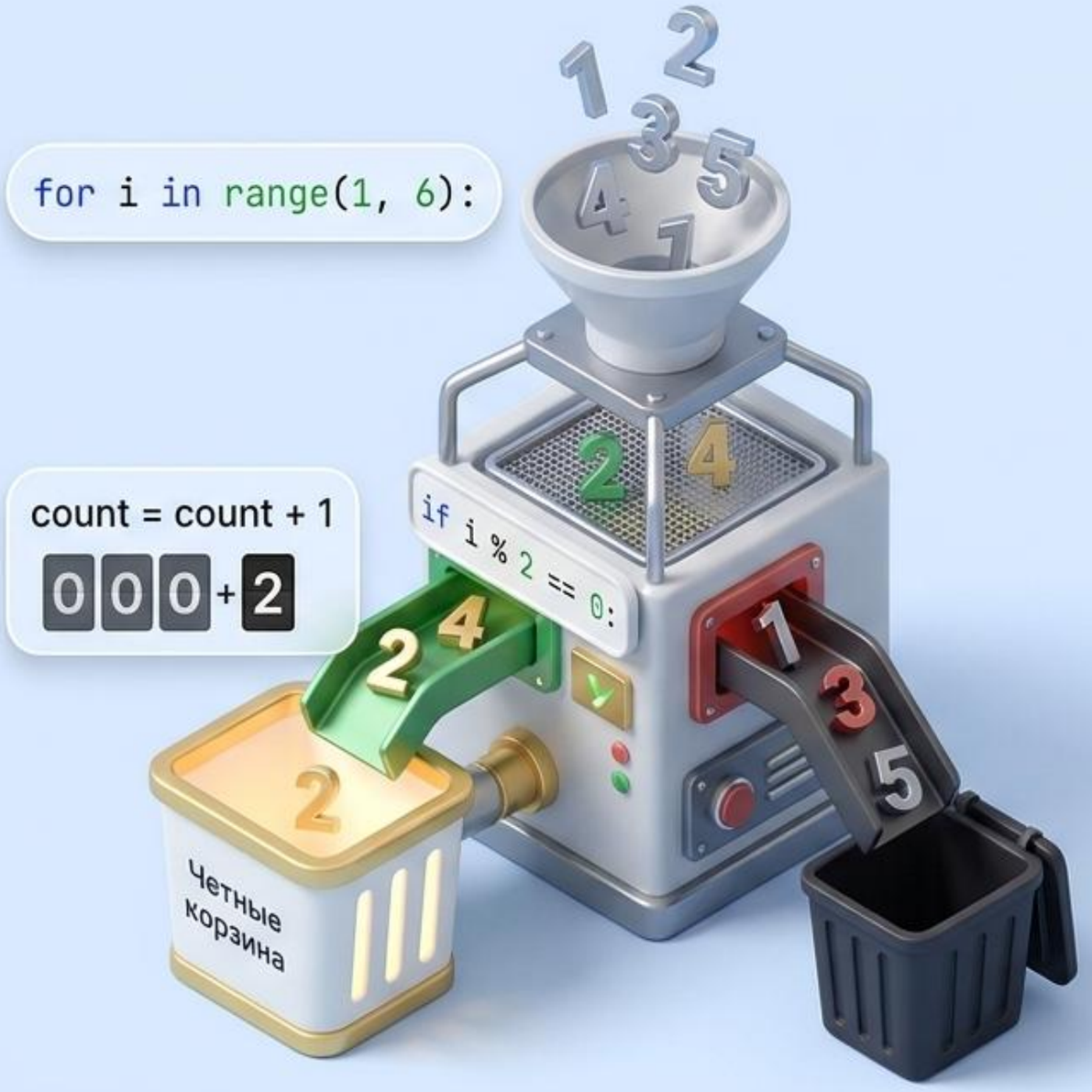



```python
for i in range(5):
    if i == 2:
        continue  # Пропускаем двойку
    print(i)
```

(Вывод: 0, 1, 3, 4)

Собираем всё вместе: Считаем четные числа

iSMART



1. **Подготовка:**
   `count = 0`
   (Ставим пустую корзину)
2. **Запуск конвейера:**
   `for i in range(1, 6):`
   (Перебираем от 1 до 5)
3. **Фильтр:**
   `if i % 2 == 0:`
   (Остаток от деления на 2 равен нулю?)
4. **Действие:**
   `count = count + 1`
   (Кладем в корзину)

В корзине окажутся числа 2 и 4. count будет равен 2.

Памятка программиста

Конструкция:	Что делает:
`range(N)`	Перебирает числа от 0 до N-1.
`while условие:`	Крутится, пока условие Истина.
`total = total + x`	Накопитель (собирает сумму).
`break`	Аварийный выход из цикла.
`continue`	Перемотка на следующий шаг.



Цикл – это инструмент, без которого невозможна ни одна серьезная программа!